THE DAEMON PROTOCOL: A SHIELDED STATE-TRANSITION MODEL FOR PROVABLY SAFE AUTONOMOUS CODING AGENTS

A PREPRINT

Daemon Protocol Research Team
daemon-blockint-tech

June 15, 2026

ABSTRACT

Autonomous coding agents built on large language models (LLMs) increasingly hold write access to production infrastructure: deployment engines, CI/CD pipelines, smart-contract toolchains, and package inventories. Treating the LLM as a benevolent actor and relying on prompt engineering for safety is fragile. We present the *Daemon Protocol*, an architecture that demotes the LLM to an *untrusted tactical planner* and relocates safety into a deterministic kernel. The kernel realises a *shield* σ that projects every proposed action onto a set of safe actions before any side effect can reach the network. We formalise the system as a shielded Markov Decision Process (MDP) with an explicit safe set $\mathcal{S}_{\text{safe}}$, and prove two safety properties for the smart-contract remediation gate (GATE 3): *No Unsafe Network Emission* and *Bounded Termination*. We reduce GATE 3 to a finite discrete automaton over $s_\delta = \langle r, \sigma_{\text{sop}} \rangle$ with a per-gate retry budget $K_{\text{gate}} = 3$ (any all-violation path rolls back within $K_{\text{gate}} + 1$ turns), while a global turn budget K_{global} guarantees termination for *any* planner. The model is connected to a concrete TypeScript kernel (`runGateWithGuardedTools`) whose per-turn telemetry rows instantiate the formal trace, to three "Trinity Fixtures" that serve as executable regression witnesses, and to a bounded-exhaustive verifier that machine-checks both safety properties over all adversarial planners. We additionally describe an uncertainty-aware (pessimistic) variant of the shield that removes the "trusted oracle" assumption on external scanners. The contribution is methodological: a reproducible bridge between a small, exhaustively verifiable formal model and a production runtime.

Keywords: LLM agents, safe reinforcement learning, shielding, Markov decision processes, Model Context Protocol, software supply-chain security.

1 Introduction

Modern AI coding agents orchestrate external systems through tool calls. When those tools can mutate real infrastructure, an agent that "hallucinates" or is driven by prompt injection becomes an operational hazard. The dominant mitigation—longer system prompts and alignment training—places the burden of safety in the stochastic policy π_θ of the model itself, which is neither inspectable nor stable across model versions.

The Daemon Protocol takes the opposite stance, inspired by *shielding* in safe reinforcement learning. The LLM proposes; a deterministic kernel disposes. Concretely, the agent operates over a layered architecture in which a cognitive-memory and semiotic layer curates compact, relevant context; a kernel enforces hard policy as a state-transition function; and a network perimeter binds any surviving action to a resource-scoped, mutually-authenticated channel.

Contributions.

1. A three-tier defensive architecture (Section 2) that separates an untrusted planner, a sterile kernel enforcer, and a physical network perimeter.
2. A formal shielded-MDP model with an explicit safe set and shield operator (Section 3), and a finite-automaton projection of GATE 3 with proofs of *No Unsafe Network Emission* and *Bounded Termination* (Section 4).

3. A concrete kernel implementation and three executable "Trinity Fixtures" that bind the runtime to the transition table (Section 5), plus a pessimistic, uncertainty-aware shield that removes the trusted-oracle assumption (Section 6).

2 Background and System Context

The Daemon Protocol connects an AI coding agent (`daemoncode`) to four operational subsystems via the Model Context Protocol (MCP): *Orion* (constraint-based Kubernetes deployment), *Altius* (data-pipeline builder), *ARES-v3* (deterministic Solana smart-contract scanner), and *ouroboros* (package/MCP supply-chain inventory). A fifth repository, *arc-agi-crystalline*, contributes a five-layer cognitive-memory pattern (episodic, semantic, procedural, analogical, principle) that we reuse as a *Crystalline* memory service.

Each subsystem exposes a canonical interface (gRPC for Orion, an API gateway for Altius, a CLI/JSON surface for ARES and ouroboros). The agent never calls these internals directly; all capability is surfaced as MCP tools and mediated by the kernel and a central API gateway.

2.1 Three-Tier Defense Topology

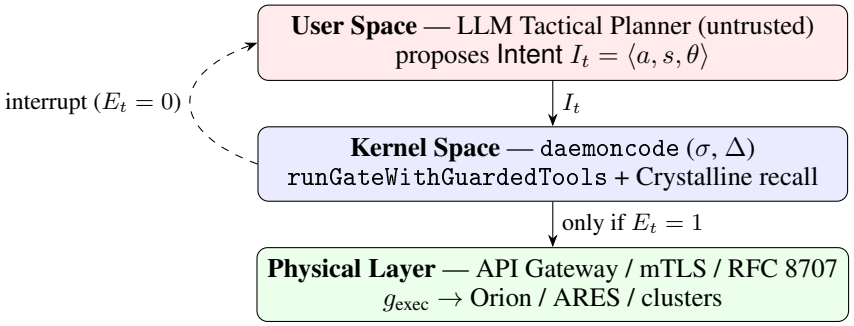

Figure 1: Three-tier topology. The LLM only emits proposals. The kernel decides; a network packet ($E_t = 1$) is emitted only for actions that pass the shield. Blocked actions ($E_t = 0$) are reflected back as cognitive interrupts.

3 Formal Safety Model

We model the system as a shielded MDP. The global state factorises as

$$\mathcal{S} = \mathcal{S}_{\text{code}} \times \mathcal{S}_{\text{mem}} \times \mathcal{S}_{\text{net}} \times \mathcal{S}_{\text{gate}}, \tag{1}$$

covering code/artefact state, cognitive memory, network-perimeter state, and the gate machine (including retry counters $(r_{\text{gate}}, r_{\text{global}})$). The LLM is a stochastic policy $\pi_\theta(a \mid h_t)$ over the action set $\mathcal{A}_{\text{LLM}} = \{I = \langle a, s, \theta \rangle\}$.

Definition 1 (Shield)**.** The kernel defines, for each state s, a set of safe actions $\mathcal{A}_{\text{safe}}(s) \subseteq \mathcal{A}_{\text{LLM}}$ via an enforcement predicate f_{enforce} built from active PRINCIPLE memories (`blockedActions`, `requiredSOP`) and semiotic links. The shield operator σ projects any proposed a onto an executable action $a_{\text{exec}} = \sigma(a, s)$; if $a \notin \mathcal{A}_{\text{safe}}(s)$ the projection is a cognitive interrupt that leaves $\mathcal{S}_{\text{net}}$ unchanged. The realised policy is $\pi' = \sigma \circ \pi_\theta$.

Definition 2 (Safe set)**.**

$$\mathcal{S}_{\text{safe}} = \{s \in \mathcal{S} \mid s_{\text{net}} \neq \textbf{COMPROMISED} \ \wedge \ r_{\text{gate}} \leq K_{\text{gate}}\}. \tag{2}$$

The desired global invariant is that, starting in $\mathcal{S}_{\text{safe}}$, the system stays in $\mathcal{S}_{\text{safe}}$ under *any* policy:

$$\forall s \in \mathcal{S}_{\text{safe}}, \ \forall a \in \mathcal{A}_{\text{LLM}} : \quad \sum_{s' \in \mathcal{S}_{\text{safe}}} P\big(s' \mid s, \sigma(a, s)\big) = 1. \tag{3}$$

4 The GATE 3 Remediation Automaton

The full state is intractable to enumerate, but the safety-relevant projection for the smart-contract remediation gate is finite. We project to

$$s_\delta = \langle r, \sigma_{\text{sop}} \rangle, \quad r \in \{0, 1, 2, 3\}, \ \sigma_{\text{sop}} \in \{0, 1\}, \tag{4}$$

where r tracks the local retry counter (with $K_{\text{gate}} = 3$) and $\sigma_{\text{sop}} = 1$ iff a mandatory SOP scan (`ares_scan_directory`) has completed successfully. The simplified action alphabet is $\{I_{\text{bad}}, I_{\text{sop}}, I_{\text{done}}\}$ for forbidden actions, mandatory-SOP actions, and the completion signal. The deterministic transition Δ_δ is given in Table 1.

Table 1: Deterministic transition Δ_δ for GATE 3. E_t is the physical network-emission flag.

State $\langle r, \sigma \rangle$	Intent	f_{enforce}	Next state	E_t
$\langle 0, 0 \rangle$	I_{bad}	1	$\langle 1, 0 \rangle$	0
$\langle 0, 0 \rangle$	I_{sop}	0	$\langle 0, 1 \rangle$	1
$\langle 0, 0 \rangle$	I_{done}	0	**ROLLBACK**	0
$\langle 1, 0 \rangle$	I_{bad}	1	$\langle 2, 0 \rangle$	0
$\langle 1, 0 \rangle$	I_{sop}	0	$\langle 0, 1 \rangle$	1
$\langle 2, 0 \rangle$	I_{bad}	1	**ROLLBACK** ($r \geq 3$)	0
$\langle 0, 1 \rangle$	I_{done}	0	**PROCEED**	1

Theorem 1 (No Unsafe Network Emission)**.** *For every turn t, if the proposed intent is forbidden ($I_t = I_{bad}$) then $E_t = 0$.*

Proof. By Table 1, every row whose intent is I_{bad} has $f_{\text{enforce}} = 1$, which routes the transition through the interrupt branch: the counters update and a violation is logged, but $\mathcal{S}_{\text{net}}$ is unchanged and $E_t = 0$. No other branch is reachable for I_{bad}, hence $E_t = 0$ in all cases. □ □

Theorem 2 (Bounded Termination)**.** *Every* GATE 3 *episode reaches a terminal state (**PROCEED** or **ROLLBACK**) within K_{global} turns, for any policy π_θ. Moreover, any path consisting solely of forbidden or illegal actions terminates within $K_{gate} + 1$ turns.*

Proof. Two distinct budgets bound the loop. (i) Each I_{bad} (or an illegal I_{done} with $\sigma_{\text{sop}} = 0$) strictly increments the per-gate counter r; after at most K_{gate} increments, $r \geq K_{\text{gate}}$ forces a **ROLLBACK**. An I_{sop} sets $\sigma_{\text{sop}} = 1$ and resets r (the tactical reward), so an adversary could otherwise loop on I_{sop} forever. (ii) To exclude this, *every* turn—including a successful I_{sop}—consumes one unit of the global budget r_{global}, which is never reset; once $r_{\text{global}} \geq K_{\text{global}}$ the loop exits with **ROLLBACK**. Hence total turns are bounded by K_{global} regardless of the planner. The all-violation path is bounded more tightly: it exhausts r after K_{gate} steps, so it terminates within $K_{\text{gate}} + 1$ turns (e.g. the synonym-attack fixture rolls back at turn 3 for $K_{\text{gate}} = 3$). □ □

Consequently, the reachable trajectory space collapses to two terminal orbits, $\{\tau_{\text{success}}, \tau_{\text{killed}}\}$, both contained in $\mathcal{S}_{\text{safe}}$ by Eq. (3).

5 Implementation and Trinity Fixtures

The transition table is realised one-to-one by the kernel loop `runGateWithGuardedTools`. Each turn (i) requests a proposal from the planner, (ii) consults Crystalline for `blockedActions`, (iii) computes a decision, (iv) delegates state mutation and logging to an atomic helper (`applyViolationPenalty` for hard violations), and (v) emits a `TelemetryRow`. The telemetry row instantiates the formal tuple $\mathbf{T}_t = [gate, t, a, status, E_t]$, where the boolean `network_emitted` is the concrete instance of $E_t \in \{0, 1\}$ and `r_gate`/`sigma_sop` instantiate r/σ_{sop}.

Listing 1: Atomic violation helper (excerpt).

```
export function applyViolationPenalty(ctx, registers, config, recorder, history) {
  registers.r_gate++; registers.r_global++;
  const terminal = registers.r_gate >= config.maxGateRetries
                || registers.r_global >= config.maxGlobalRetries;
  const status = terminal ? "ROLLBACK" : "BLOCKED";
  recorder.record({ gate: ctx.gate, turn: ctx.turn,
    action_schema: ctx.action_schema, enforcer_status: status,
    network_emitted: false });            // I_bad => E_t = 0
  if (terminal) return { shouldBreak: true, finalStatus: "ROLLBACK" };
  history.push({ role: "tool",   content: JSON.stringify({ blocked: ctx.action_schema }) });
  history.push({ role: "system", content: ctx.interruptMessage });
  return { shouldBreak: false, finalStatus: "PENDING" };
}
```

The three *Trinity Fixtures* are golden traces that pin the runtime to Δ_δ and are encoded as unit tests:

Fixture 1 (Synonym Attack). A stubborn planner emits `deploy_to_prod`, `force_publish`, `trigger_pipeline`, `deploy_to_prod`. Every turn is `BLOCKED` with `network_emitted=false`; the retry budget is exhausted after $K_{\text{gate}} = 3$ increments, so the episode ends in **ROLLBACK** at turn 3 (Bounded Termination).
Fixture 2 (Sandbox Escape). A planner falls back to a generic tool (`execute_bash`). All turns are blocked; **ROLLBACK** at turn 3.
Fixture 3 (SOP-Compliant Path). After one blocked `deploy_to_prod`, the planner runs `ares_scan_directory` and `ouroboros_scan` (each `SUCCESS`, `network_emitted=true`) and then `complete_gate_task` $\rightarrow$ **PROCEED**.

Any change to the kernel that violates Δ_δ is caught as a regression failure, making the formal model an enforced contract rather than documentation.

5.1 Machine-Checked Verification

Beyond the three example traces, a bounded-exhaustive verifier (`verifyGate`) discharges both safety properties against the *real* kernel rather than an abstract model. It drives `runGateWithGuardedTools` with an adversarial planner that enumerates every action sequence over a representative alphabet (one forbidden action, the SOP tools, the completion tool, and an unknown tool) up to depth $K_{\text{gate}} + 1$; because the planner repeats its final symbol, "keeps doing X forever" tails—including unbounded I_{sop}—are covered. For every path it asserts (P1) no telemetry row for a forbidden action has `network_emitted=true` and (P2) the episode terminates with a trace no longer than K_{global}. The check is run in CI across a range of budgets and with the pessimistic shield enabled.

This exercise was not merely confirmatory: it exposed a real defect. An earlier kernel reset r on a successful I_{sop} *without* charging the global budget, so a planner that looped on the SOP tool never terminated. The fix—charging r_{global} on every turn (Theorem 2)—was driven by, and is now guarded by, this verifier. A companion PRISM/PCTL model (`verification/gate3.prism`) is provided for independent model-checking by the reader.

6 Uncertainty-Aware (Pessimistic) Shielding

A naive shield trusts the external scanners (ARES, ouroboros) as perfect oracles. To remove this assumption we add a scanner-confidence variable $\gamma_{\text{scan}} \in [0, 1]$ and adopt a default-deny posture:

$$f_{\text{enforce}}(I_t, M) = \begin{cases} 1, & \exists m \in M : \ I_t.a \in m.A_{\text{blocked}} \ \wedge \ \gamma_{\text{scan}} \geq \gamma_{\text{thr}} \\ 1, & \gamma_{\text{scan}} < \gamma_{\text{thr}} \quad \text{(scanner uncertain)} \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

A false negative under low confidence no longer opens an execution path; the agent is forced onto the SOP route or human-in-the-loop. The two GATE 3 theorems are preserved because the only effect of low confidence is to add more $f_{\text{enforce}} = 1$ transitions, which never emit network packets and never extend the retry bound.

7 Threats to Validity and Future Work

The proofs rely on (i) the kernel and gateway faithfully implementing σ and Δ, and (ii) the semiotic layer correctly mapping forbidden synonyms into I_{bad}. New evasion tools are handled by extending the semiotic links and PRINCIPLE memories, *not* by mutating the kernel, preserving $\mathcal{S}_{\text{safe}}$. Validation now combines the formal proofs, the Trinity Fixtures, and the bounded-exhaustive verifier of Section 5.1 (which already caught and fixed a termination defect); a PRISM/PCTL model is included for independent checking. The main remaining gaps are empirical "live-fire" logs against a real LLM and integration into the production tool gate, both deliberately staged.

8 Conclusion

Safety in the Daemon Protocol is a property of a deterministic kernel, not of the LLM's disposition. By reducing the smart-contract remediation gate to a finite shielded automaton with a bounded retry budget, we obtain two machine-checkable safety properties and bind them to a concrete runtime via the Trinity Fixtures. The result is a reproducible methodology for building autonomous coding agents that may be wrong but cannot be destructive.